

SE333 MCP Testing Agent Reflection

Tasnim Zayed

November 16, 2025

Abstract

This project implements an intelligent testing agent using the Model Context Protocol (MCP) and FastMCP. The agent integrates with VS Code to generate JUnit tests, run Maven and JaCoCo, and automate Git workflows. For the provided sample Maven project, the agent achieved full line, method, and class coverage (100%), demonstrating how MCP tools can support automated, iterative test generation and continuous feedback.

1 Introduction

The goal of this project was to design and implement an MCP-based testing assistant that can:

- Inspect a Java project and identify test targets
- Generate and refine JUnit tests
- Run Maven and JaCoCo to measure coverage
- Automate common Git actions as part of a testing workflow
- Work interactively inside VS Code as an AI-assisted development tool

The submitted repository contains an MCP server (`server.py`), a sample Maven project with 100% test coverage, comprehensive documentation, and screenshots demonstrating the workflow end-to-end.

2 MCP Tools Implemented

The MCP server exposes 14 tools organized into key categories:

2.1 Test & Coverage Tools

- `list_java_methods` – Scan Java files for candidate methods
- `generate_basic_junit` – Generate baseline JUnit tests
- `mvn_test`, `mvn_coverage` – Run tests and JaCoCo
- `parse_jacoco`, `coverage_summary` – Parse coverage XML and summarize results
- `find_uncovered_methods`, `iterate_coverage` – Identify gaps and iterate towards higher coverage

2.2 Git Automation Tools

- `git_status` – Return repository status
- `git_add_all` – Stage all changes
- `git_commit` – Create commits with messages
- `git_push` – Push to remote repository
- `git_pull_request` – Create pull requests

2.3 Utilities

- `try_fix_known_bugs` – Fix common issues
- Helper functions used internally by coverage iteration

Screenshots from VS Code show these tools available in the MCP tool list and invoked in chat interactions.

3 Coverage Results

3.1 Initial State

Before generating tests with the MCP agent, the target class (`Calc`) had no test coverage (0% line and method coverage).

3.2 Final State

After using the MCP tools to generate and run tests, the following coverage was achieved:

- **Instruction Coverage:** 100.0% (7/7)
- **Line Coverage:** 100.0% (2/2)
- **Complexity Coverage:** 100.0% (2/2)
- **Method Coverage:** 100.0% (2/2)
- **Class Coverage:** 100.0% (1/1)

The JaCoCo HTML report (included in screenshots) confirms that all lines and methods in `Calc` are covered by `CalcTest`.

4 Agent Workflow and Iteration

The MCP agent supports an iterative workflow inside VS Code:

1. Run `list_java_methods` to detect methods without tests
2. Call `generate_basic_junit` to create starter tests
3. Use `mvn_test` and `mvn_coverage` to execute tests and regenerate JaCoCo reports

4. Call `coverage_summary` and `find_uncovered_methods` to see remaining gaps
5. Use Git tools to stage, commit, and push changes when coverage improves

This mirrors how an AI assistant could continuously nudge a codebase towards better coverage through automated iteration.

5 Challenges and Solutions

5.1 MCP Server Integration

Understanding the Streamable-HTTP transport and why visiting the MCP URL in a browser returns an error. I resolved this by confirming that the server is meant for MCP clients (VS Code), not direct browser use. The URL `http://127.0.0.1:6060/mcp` is an API endpoint, not a website.

5.2 Environment Setup

Activating the correct virtual environment and running Python and Maven consistently on Windows required some trial and error. Once fixed, I documented the exact commands in the README with Windows-specific instructions using PowerShell.

5.3 JaCoCo Parsing

The XML report contains multiple nested counters at different levels (package, class, method). I implemented parsing logic that focuses on the top-level project coverage totals to avoid duplicate entries, ensuring accurate representation of overall coverage metrics.

6 Lessons Learned

- Tooling around tests (MCP, Maven, JaCoCo, Git) can be automated and chained together reliably
- Even a simple project benefits from an automated “generate–run–measure” loop
- Careful logging, screenshots, and metrics collection are essential for explaining and grading an engineering project, not just running code
- Prompt engineering for the AI agent significantly impacts test quality and coverage improvement
- Iterative improvement through automated tools is more effective than single-pass generation
- Integration with existing development tools (VS Code, Maven) makes AI agents practical for real workflows

7 Project Metrics

The following metrics were collected using an automated script (`get_my_metrics.py`):

```
=====
PROJECT METRICS FOR REFLECTION
=====
```

- ```
1) MCP TOOLS COUNT
 Total MCP tools: 14

2) COVERAGE STATISTICS (JaCoCo)
 INSTRUCTION: 100.0% (7/7)
 LINE: 100.0% (2/2)
 COMPLEXITY: 100.0% (2/2)
 METHOD: 100.0% (2/2)
 CLASS: 100.0% (1/1)

3) TEST FILES
 Total Test Classes: 1
 - CalcTest.java

4) GIT COMMITS
 Total Commits: 12
 Recent commits:
 - c92b925 docs: add Quickstart to README
 - 3f82148 docs: add Highlights to README
 - f12bab3 chore: add CI badge to README
 - c244cd4 chore: add CI badge to README
 - 096f1b4 chore: add PR template

5) CODE STATISTICS
 server.py: Contains 14 MCP tools
 Test Coverage: 100% across all metrics
```

```
=====
```

## 8 Conclusion

This MCP Testing Agent demonstrates how AI-assisted tools can plug directly into a developer workflow: listing methods, generating tests, running coverage, and managing Git operations from inside an editor. While the provided sample is intentionally small (the `Calc` class), the same pattern scales to larger Java projects and more advanced test generation strategies.

The project successfully achieved:

- Full automation of the test generation workflow through 14 MCP tools
- 100% code coverage improvement from 0% baseline
- Seamless integration with existing development tools (VS Code, Maven, Git)
- Extensible architecture for future enhancements

- Automated Git workflow integration (status, commit, push, PR creation)

Future work could include support for more complex test scenarios, integration with additional testing frameworks (TestNG, Mockito), enhanced AI-driven test case generation based on code analysis and historical bug patterns, and support for mutation testing to validate test quality beyond coverage metrics.